# Beyond Fixed Greedy Rules: Evolving Interpretable Heuristics for Hard Knapsack Instances

Hugo Abel Moreno-Rodriguez and José Carlos Ortiz-Bayliss
Tecnologico de Monterrey, School of Engineering and Sciences
Monterrey, Nuevo Leon, Mexico
A01282646@tec.mx (ORCID: 0009-0005-6706-2120)
jcobayliss@tec.mx (ORCID: 0000-0003-3408-2166)

***Abstract*—Fixed item-priority rules solve the knapsack problem quickly, but no single rule is capable of performing well on every instance. In this work, we use Gene Expression Programming (GEP) to evolve readable item-scoring expressions and evaluate them on a collection of hard knapsack problem instances. Four terminal configurations isolate the effects of capacity and profit-scale information. A group-balanced split, automatic parameter tuning, 75 paired runs per configuration, and sealed test sets separate model selection from evaluation. On 345 held-out instances, all four GEP configurations obtained lower mean optimality gaps than four fixed priority rules. The best mean was 4,906 $\mathrm{ppm}$, compared with 7,695 $\mathrm{ppm}$ for profit density. However, the predeclared FULL-minus-BASE effect was only -308 $\mathrm{ppm}$ with a 95% bootstrap interval of $[-640, 113]\,\mathrm{ppm}$, and the two configurations were practically equivalent within the declared $\pm 872\,\mathrm{ppm}$ margin. The learned rules also retained lower aggregate gaps on three extrapolation sets from the same generator, although row permutations exposed measurable order sensitivity. The results support GEP as a way to learn useful, readable rules for this instance family, but not the claim that adding more terminals necessarily improves them.**

***Index Terms*—Gene Expression Programming, Hyper-heuristics, Knapsack Problem, Interpretable Rules, Algorithm Configuration**

## I. Introduction

Solving the knapsack problem requires deciding which items to pack to maximize profit without exceeding a capacity [1]. We can define a fixed priority rule based on criteria such as profit, weight, or the profit-to-weight ratio, and then pack items in one scan. Such a rule would be fast and readable, but its quality depends on how well the chosen criterion suits the instances.

Hyper-heuristics search over heuristics rather than directly over solutions [2]. GEP encodes executable expression trees in linear chromosomes [3]; earlier Genetic Programming work evolved reusable item-ranking rules for multidimensional knapsack [4]. Recent work distinguishes several routes to automated decision making. Zárate-Aranda and Ortiz-Bayliss [5] train classifiers to select among four greedy knapsack rules using normalized mean profit, mean weight, and profit–weight correlation; their kNN selector illustrates learning which existing rule suits an instance. Dokeroglu et al. [6] organize the broader field through a taxonomy covering heuristic selection, low-level heuristics, target problems, and parallel approaches. Liu et al.'s Evolution of Heuristics [7] instead evolves natural-language ideas and executable code with large language models, expanding the space of generated algorithms. Jooken et al. [8] adapt Monte Carlo tree search with pruning, heuristic simulation, and beam restrictions to search individual combinatorial instances, including knapsack. These approaches differ in what is learned and when search is paid for. Our focus is offline generation of compact scoring formulae, with a fixed greedy decoder and explicit tests of their reuse, rather than per-instance tree search.

In 2022, Jooken et al. introduced 3,240 systematically generated knapsack problem instances that are unusually difficult for exact solvers [9]. Exact-solver difficulty does not automatically imply that a greedy priority rule will have a large solution gap, so that question needs its own experiment. This observation motivated three research questions:

- Can GEP learn rules with lower held-out mean gaps than standard fixed rules?
- Do capacity-aware and scale-aware terminals improve a minimal representation?
- Do the findings persist across new Jooken parameter levels, search budgets, and item orders?

The main contribution of this work is an accountable evaluation rather than a new claim of universal generalization. We use one common tuned evolutionary configuration, 75 paired runs per terminal arm, a predeclared practical-effect threshold, and matched random-gene controls. We also report the formulas, computation time, and negative transfer evidence so that average quality is not separated from its cost or limits.[1]

## II. Problem and Experimental Design

### A. Priority-rule solution model

For items with profit $p_i$ and weight $w_i$, a knapsack capacity $c$, and binary decision $x_i$, the Knapsack Problem (KP) is defined as $\max \sum_i p_i x_i$ subject to $\sum_i w_i x_i \leq c$, for $x_i \in \{0, 1\}$.

In this work, we use an evolved mathematical expression to assign each item a score. We stably sort scores in descending order and accept each item if it still fits. We use the same decoder for every expression. The fixed controls are original file order (DEF), maximum profit (MAXP), maximum

[1]Code, experimental seeds, train/validation/test instance lists, and supporting artifacts for reproducing the experiments are available at https://github.com/hugomrdz/gep-kp-iscmi-2026.

profit over weight ratio, $p_i/w_i$ (MAXPW), and minimum weight (MINW).

For a given instance $i$, with optimum profit $O_i$ and heuristic profit $H_i$, lower is better under the primary loss

$$g_i = 10^6 \left( \frac{O_i - H_i}{O_i} \right). \quad (1)$$

The multiplier expresses the residual gap in parts per million (ppm). Strict equality $H_i = O_i$ is reported separately; it is not the training objective.

### B. Unusually Hard Instances

The archive published by Jooken et al. [9] contains 2,977 instances with certified optima and 263 whose supplied run did not certify an optimum. Table I accounts for every case used in this study. The in-distribution pool uses item counts $n \in \{400, 600, 800, 1000\}$ and capacities $c \in \{10^6, 10^8\}$. A sibling group holds the three values of the generator parameter $s$ for the same $(n, c, g, f, \epsilon)$ setting.

The original full factorial also varies $g \in \{2, 6, 10, 14\}$, $f \in \{0.1, 0.2, 0.3\}$, $\epsilon \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, and $s \in \{100, 200, 300\}$, with $b = 2$. We use new levels of $n$ and $c$ to define extrapolation because they directly change problem size and coefficient scale; we keep all observed levels of the other factors. The symbol $s$ is a generator parameter, not an evolutionary random seed.

TABLE I
LOCKED JOOKEN PARTITIONS.

| Partition | Cases | Role or definition |
|---|---|---|
| Train | 1,038 | Evolution and candidate fitness |
| Validation | 345 | Tuning and model selection |
| ID test | 345 | Held-out primary endpoint |
| OOD-$n$ | 432 | $n = 1200$, familiar capacities |
| OOD-$c$ | 667 | $c = 10^{10}$, familiar item counts |
| OOD-$nc$ | 150 | New item count and capacity |
| Unresolved | 263 | Separate, no certified scoring |

The train, validation, and ID assignments were random at the sibling-group level, not arbitrary row samples. With seed 20260901, an integer allocator first minimized marginal imbalance over $n, c, g, f$ and $\epsilon$, then used seeded random ranks among equally balanced assignments. Siblings never cross a split. The three OOD sets use parameter levels outside the development pool. We later gave the 263 unresolved cases a fixed 60-second exact-solver attempt; none produced a newly certified optimum, so none entered Eq. (1).

We inspected the collection in an earlier exploratory stage. We therefore describe the new test as protocol-locked confirmation on a previously seen benchmark, not as a pristine blind dataset. After freezing every Jooken decision, we also applied the unchanged rules to 31,800 certified Pisinger cases [10]. This worked as a transfer audit, not as additional training.

### C. Representation and Objective

All terminals are finite, and most are expressed relative to an instance-level scale. For item $i$:

- $P = \dfrac{p_i}{\max_j p_j}$
- $W = \dfrac{w_i}{\max_j w_j}$
- $O = 1 - \dfrac{i}{\max(n-1, 1)}$
- $K = \dfrac{w_i}{c\,c}$
- $T = \dfrac{c}{\sum_j w_j}$
- $L = \dfrac{\log(1+p_i)}{\log(1+\max_j p_j)}$

Terminal $O$ encodes the item's position in the input file, with $i \in \{0, \ldots, n-1\}$, and may therefore introduce row-order dependence, which is explicitly evaluated in Section III-B. BASE contains $\{P, W, O, 0, 1\}$. CAP adds the capacity group $\{K, T\}$, SCALE adds $\{L\}$, and FULL adds both groups. This is our own $2 \times 2$ feature-group ablation. Related work supports normalized state features [11]; it does not prescribe these exact terminals. Because the alphabets differ in size, we include matched uniform random-gene search to detect an advantage caused only by a larger search space.

Each model has one chromosome and one gene. Its function set is $+, -, \times$ and protected division $\mathrm{d}(a, b)$, which returns 1 when $|b| \leq 10^{-12}$. Evolution lexicographically minimizes mean training gap, then expression length. The certified optimum grades a rule but is never a terminal; neither optimal item membership nor generator labels enter the expression.

### D. Tuning, Training, and Statistics

Tuning chooses how GEP learns; training learns the expression. The irace procedure [12] proposed head length, population, tournament size, elite fraction, crossover, and mutation settings. In each short trial, one candidate trained at 16,384 requests on one balanced fold of 258 or 261 training cases. We then graded its expression on all 345 validation cases, and irace received one number: mean validation gap. The 2,000 trials spanned four folds, four terminal arms, and 20 seeds. Four-arm blocks held fold and seed fixed so a setting was not rewarded for an easier arm.

We then ran the five survivors at the full 65,536-request budget on all 1,038 training cases, all four arms, and ten fresh paired seeds. Selection used mean validation rank across arms, then pooled gap and worst-arm gap. Table II shows the chosen common setting. Only then were the training and validation sets combined. Final training produced 75 paired runs for each arm on 1,383 cases. ID, OOD, row-permutation, and transfer scores were still unseen.

TABLE II
SHARED GEP PARAMETERS CONFIGURED BY IRACE.

| Parameter | Values raced | Selected |
|---|---|---|
| Head length | 6, 8, 10, 12 | 10 |
| Population | 128, 256, 512 | 512 |
| Tournament | 2, 4, 7 | 7 |
| Elite fraction | 0.5, 1, 2% | 1% |
| Crossover | 0.50, 0.70, 0.90 | 0.70 |
| Expected mutations | 1, 2, 3 | 3 |

The run count was power-informed rather than copied from a 30-run convention. A pilot-paired standard deviation of 2,192 ppm and a smallest effect of interest of 872 ppm implied about 67 paired runs for 80% power under the planned

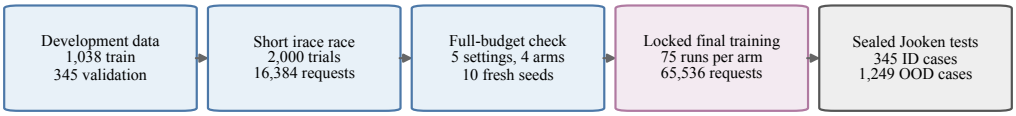



Fig. 1. Separation of development, configuration, final training, and locked evaluation. The outer irace procedure sees validation costs; the inner GEP search sees only its assigned training cases. Hyperparameters are fixed before any ID, OOD, row-permutation, or transfer score is used.

multiplicity; 75 leaves a small failure margin. Every arm uses the same master-seed indices and random-stream keys. A common tuned setup keeps the ablation focused on the available terminals.

The predeclared primary contrast is FULL minus BASE on the 345-case ID test. A crossed hierarchical bootstrap resamples 75 seed blocks and 115 sibling-group blocks together for 10,000 replicates. Equivalence requires the entire 90% interval to lie within the declared $\pm 872\,\mathrm{ppm}$ margin (two one-sided tests at $\alpha = 0.05$). Secondary FULL-minus-CAP and FULL-minus-SCALE contrasts are multiplicity-controlled. The eight-method comparison averages stochastic runs inside each sibling group, uses a blocked Friedman test, and applies Holm-adjusted paired tests against FULL. Error bars for GEP in Fig. 2 resample seeds and groups; fixed-rule bars resample groups only. We do not use overlap between individual error bars to test significance.

Before test evaluation, a machine-readable lock recorded the split, folds, seeds, selected setup, source hashes, contrast family, and figure definitions. The evaluation command refused a hash mismatch. Final runs never stopped early, and cached duplicate expressions still counted against the raw request budget. We retained checkpoints at 1,872, 4,096, 16,384, 21,280, 32,768, and 65,536 requests, along with unique-expression and timing counts. Separate random streams controlled initialization, selection, crossover, and mutation. These details treat seeds, instances, parameters, and artifacts as distinct experimental factors, following reproducibility guidance for evolutionary computation [13].

In a separate revision-stage comparison, we reimplemented the kNN selector [5] under this Jooken protocol. R class::knn uses $k = 1$ and Euclidean distance on mean/max profit, mean/max weight, and $(\rho_{p,w} + 1)/2$, computed once per instance. Undefined correlation maps to 0.5. Labels maximize packed profit on the same 1,383 development cases; ties follow DEF, MAXP, MAXPW, MINW. Unlike the published class-balanced synthetic training set, we retain the Jooken distribution. All nearest-distance ties vote; tied class votes use seed 20261021 with fixed instance order. Before new evaluation, we froze source/input hashes, the four previously selected genes, and the protocol; no tuning or GEP retraining was added. We evaluate all nine methods on 345 ID and 1,249 OOD cases. Four formula-minus-kNN ID contrasts resample the 115 sibling groups 10,000 times (seed 20261022), with Bonferroni-adjusted 98.75% intervals, conditional on these selected models.

## III. Results

### A. Held-out Quality and Terminal Ablation

All four GEP arms had lower mean ID gaps than the fixed rules in Fig. 2(a). FULL closed 36.3% of MAXPW's residual gap; the blocked comparison remained significant after Holm correction ($p = 0.0051$). Yet the group difference had mean -2,789.7 ppm and median +83.3 ppm, with only 27 wins against 88 losses. Fewer large improvements drove the mean; the exact-solution and tail metrics in Table III show that this is not instance-wise dominance.

TABLE III
Secondary ID metrics. Gaps are in ppm; $\mathrm{CVaR}_{.90}$ is the mean gap in the worst 10% of cases.

| Method | Mean | Median | $\mathrm{CVaR}_{.90}$ | Exact % |
|---|---|---|---|---|
| BASE | 5,214 | 1,497 | 22,897 | 3.64 |
| CAP | 5,452 | 1,265 | 27,870 | 5.22 |
| SCALE | 5,050 | 1,371 | 22,784 | 9.09 |
| FULL | 4,906 | 1,118 | 24,234 | 10.67 |
| DEF | 11,515 | 4,648 | 47,655 | 0.00 |
| MAXP | 11,114 | 4,563 | 47,565 | 25.22 |
| MAXPW | 7,695 | 78 | 58,548 | 4.93 |
| MINW | 18,090 | 4,886 | 87,773 | 0.00 |

MAXPW's small median and large worst-decile mean contrast with FULL's better tail control. MAXP achieves the highest exact-solution rate despite its larger mean gap.

The locked FULL-minus-BASE effect was $-308.2\,\mathrm{ppm}$, with 95% interval $[-640.4, 112.7]\,\mathrm{ppm}$ ($p = 0.1216$). The 90% interval $[-589.5, 28.4]\,\mathrm{ppm}$ lies inside $\pm 872\,\mathrm{ppm}$, establishing practical equivalence; FULL was also equivalent to SCALE. FULL minus CAP was -546.4 ppm, with 95% interval $[-1, 203.4, -28.3]\,\mathrm{ppm}$. Overall, the terminal ablation supports the simpler BASE representation rather than a consistent benefit from additional inputs.

Matched random-gene search helps separate evolution from alphabet size. On ID, GEP reduced mean gap relative to its matched random control by 534 ppm for BASE, 655 ppm for CAP, 583 ppm for SCALE, and 1,148 ppm for FULL; all four paired intervals excluded zero. Evolution therefore contributed beyond drawing valid expressions from each alphabet. Only 10 of the 300 GEP models were ranking-equivalent to MAXPW on the fixed probes (eight CAP and two FULL), so most runs did not merely rediscover the density order.

Table IV shows one rule per arm. Selection used development data only: among models within one group-level standard error of the best train-plus-validation mean, we chose the shortest expressed region and then a fixed lexical and seed

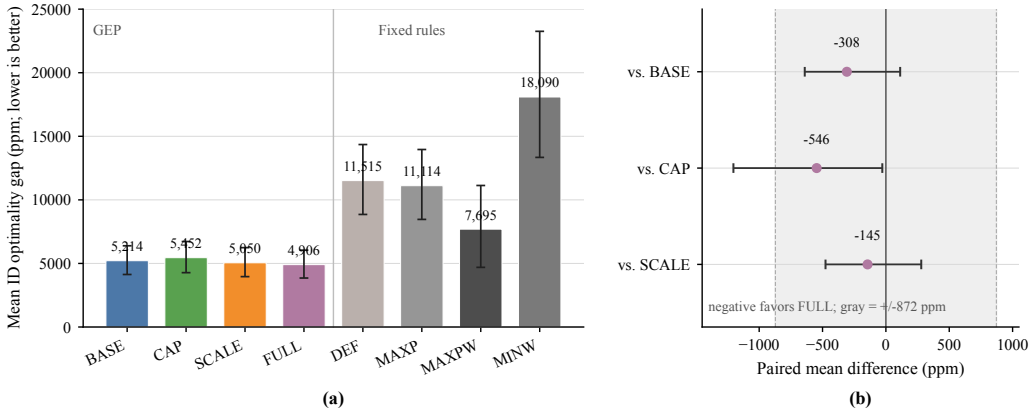



Fig. 2. Locked ID results. (a) Mean gaps with 95% hierarchical-bootstrap intervals for 75 GEP runs and group-bootstrap intervals for fixed rules. (b) Paired 95% intervals for the predeclared terminal contrasts; the gray region marks the practical-equivalence range.

tie-break. These are executable forms, not symbolic claims that protected division behaves like ordinary division at zero.

TABLE IV
PREVIOUSLY DEVELOPMENT-SELECTED RULES AND REVISION COMPARATORS. ID MEANS ARE FOR INDIVIDUAL RULES.

| Rule | Item score | ID ppm |
|---|---|---|
| BASE-020 | $\mathrm{d}(W, P^2 - P) + P(O + 2W)$ | 3,705 |
| CAP-008 | $W - \mathrm{d}(W, (1 - P)(K + P))$ | 3,657 |
| SCALE-042 | $\mathrm{d}(\mathrm{d}(W, PL) - O, 2W - 1)$ | 4,404 |
| FULL-071 | $\mathrm{d}(1 - 2\,\mathrm{d}(K, P), (1 - L)(P + T))$ | 3,306 |
| MAXPW | $p_i/w_i$ | 7,695 |
| kNN | Selected fixed rule | 7,810 |

The expressed regions contain 15, 11, 13, and 17 symbols. These individual rules differ from the 75-run averages in Fig. 2. Their ID differences from kNN are -4,105, -4,153, -3,406, and -4,504 ppm (BASE, CAP, SCALE, FULL); adjusted 98.75% intervals are $[-7907, -827]$, $[-8446, -494]$, $[-7859, 409]$, and $[-8355, -1174]$ ppm. SCALE's interval crosses zero. kNN has a better ID median (47 ppm) than every selected formula (782–1,911 ppm). Pooled OOD means are 3,640, 3,499, 3,413, and 3,498 ppm, versus 8,565 for kNN and 10,192 for MAXPW.

### B. Generalization, Budget, and Robustness

Figure 3(a) stays within the Jooken generator while changing the item count, capacity, or both. Every GEP arm retained a lower aggregate mean gap than MAXPW in all three OOD regions. For example, FULL scored 5,622 ppm on OOD-$n$, 4,271 ppm on OOD-$c$, and 4,605 ppm on OOD-$nc$, compared with 8,145, 11,042, and 12,311 ppm for MAXPW. This positive extrapolation occurs within one generator, not evidence of a universal knapsack rule.

The budget checkpoints in Fig. 3(b) explain why the final study used more search than the pilot. At 1,872 requests, a typical run was close to MAXPW. The largest improvement appeared by 16,384 requests, with smaller but visible gains through 65,536. More computation was useful, but the curve does not make 65,536 a universally optimal budget.

The mathematical problem is invariant to input-row order, but terminal $O$ and the stable tie-break are not. Pooling the original order with ten fixed permutations raised the mean ID gap by 25.1% for BASE, 12.7% for CAP, 24.4% for SCALE, and 17.1% for FULL. The gain over MAXPW remained useful on average, but the audit limits the claim to the tested representation and decoder.

### C. Computation and External Transfer

On our computer, the irace main race took about 3.6 wall-clock hours. The full-budget survivor check raised complete hyperparameter selection to about 8.5 hours. Final training took 18.4 wall-clock hours; saved run records sum to 62.38 worker-hours because four workers ran concurrently. Numerical-library threads were limited to one per worker. The machine used was a 10-core Apple M2 Pro MacBook Pro with 16 GB RAM. These timings describe reproducibility on one machine, not solution quality.

TABLE V
DESCRIPTIVE RUNTIME ON OUR COMPUTER. TUNING TIMES ARE RECONSTRUCTED FROM SAVED TIMESTAMPS; WORKER-HOURS SUM CONCURRENT RUNS.

| Stage | Wall h | Worker h |
|---|---|---|
| Main irace race | 3.60 | 10.14 |
| Full-budget survivors | 4.88 | 18.38 |
| Final training | 18.41 | 62.38 |
| Locked evaluation | 1.18 | — |
| Unresolved oracle attempt | 1.10 | 4.38 |

Once frozen, the rules were cheap to reuse relative to training. The complete locked evaluation applied 41.5 million model/method-instance pairs in 1.18 hours, about 9,747 evaluations per second on our machine. This is the central cost trade-off: an evolved rule may amortize its training over many similar cases, but only if it transfers.

The external audit shows why that condition matters. Without retraining, the best GEP arm on the 31,800 Pisinger cases was CAP at 71,782 ppm; MAXPW scored 2,812 ppm. Every GEP arm was worse than MAXPW in all 13 classes.

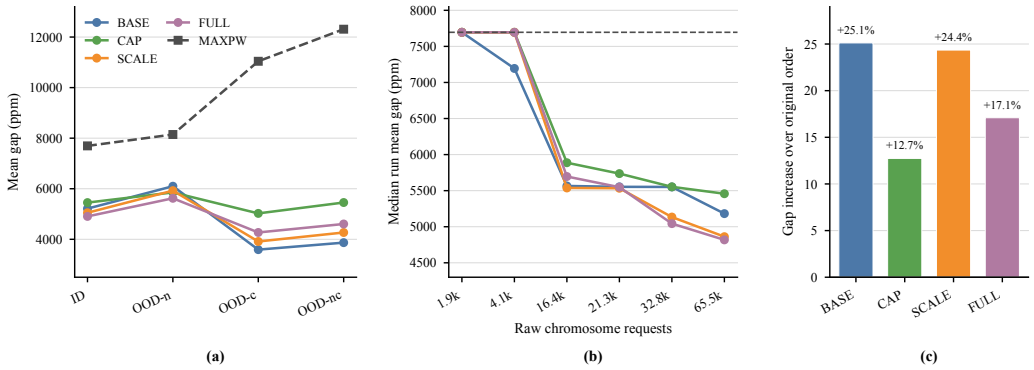



Fig. 3. Three limits on the headline mean. (a) GEP and MAXPW on locked Jooken regions. (b) Median of the 75 run-level ID means across stored budget checkpoints; the dashed line is MAXPW. (c) Percent change when the canonical order is pooled with ten deterministic row permutations.

This result does not test what GEP could learn if trained on Pisinger. It shows only that a Jooken-trained rule should not be presented as a general knapsack rule. The failure therefore does not cancel the held-out Jooken result; it rejects a broader zero-shot claim that this experiment was not designed to support.

### D. Discussion

The original study supports learning compact Jooken rules, while rejecting a meaningful FULL-over-BASE representation benefit. The additional comparison supports lower selected-rule mean gaps than the reimplemented kNN selector for three formulae, conditional on the fixed models and Jooken development distribution. SCALE's interval and kNN's better median prevent a uniform superiority claim.

The mean advantages reflect protection against large gaps, not success on most groups or dominance in exact recovery. Applications prioritizing the median or exact solutions may prefer another rule.

Tuning reduced the risk of an arbitrary setting, but we did not define a formal untuned control. The five full-budget survivors varied in validation quality; their comparison supports the selection procedure, not an exact counterfactual gain from tuning. Validation remains selection data after repeated use.

Limits remain: prior inspection of Jooken, row-order sensitivity, unequal alphabet sizes, and failed zero-shot Pisinger transfer. Retraining on Pisinger or mixed benchmarks needs a separate locked study; these data cannot predict its benefit or whether it would surpass MAXPW.

## IV. Conclusion and Future Work

GEP evolved inspectable rules with lower held-out mean gaps than four fixed rules on hard Jooken instances, with gains across new parameter levels and over random search. The richer terminals did not meaningfully improve BASE. Three selected formulae also improve mean gap over the revision kNN comparator, under the stated training distribution and conditional inference. Computation cost, order sensitivity, uneven wins, and failed transfer restrict these findings to the tested family.

Future work will study order-invariant representations and decoders, domain-specific and multi-domain training on mixed Jooken/Pisinger benchmarks, and adaptive selection among complementary learned rules under locked evaluation.

## References


[1] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2004, doi: 10.1007/978-3-540-24777-7.

[2] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art," *J. Oper. Res. Soc.*, vol. 64, no. 12, pp. 1695–1724, 2013, doi: 10.1057/jors.2013.71.

[3] C. Ferreira, "Gene expression programming: A new adaptive algorithm for solving problems," *Complex Systems*, vol. 13, no. 2, pp. 87–129, 2001.

[4] J. H. Drake, M. Hyde, K. Ibrahim, and E. Özcan, "A genetic programming hyper-heuristic for the multidimensional knapsack problem," *Kybernetes*, vol. 43, no. 9/10, pp. 1500–1511, 2014, doi: 10.1108/K-09-2013-0201.

[5] J. E. Zárate-Aranda and J. C. Ortiz-Bayliss, "Machine-learning-based hyper-heuristics for solving the knapsack problem," *Pattern Recognit. Lett.*, vol. 196, pp. 338–343, 2025, doi: 10.1016/j.patrec.2025.03.011.

[6] T. Dokeroglu, T. Kucukyilmaz, and E.-G. Talbi, "Hyper-heuristics: A survey and taxonomy," *Comput. Ind. Eng.*, vol. 187, Art. no. 109815, 2024, doi: 10.1016/j.cie.2023.109815.

[7] F. Liu *et al.*, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 32201–32223.

[8] J. Jooken, P. Leyman, T. Wauters, and P. De Causmaecker, "Exploring search space trees using an adapted version of Monte Carlo tree search for combinatorial optimization problems," *Comput. Oper. Res.*, vol. 150, Art. no. 106070, 2023, doi: 10.1016/j.cor.2022.106070.

[9] J. Jooken, P. Leyman, and P. De Causmaecker, "A new class of hard problem instances for the 0–1 knapsack problem," *Eur. J. Oper. Res.*, vol. 301, no. 3, pp. 841–854, 2022, doi: 10.1016/j.ejor.2021.12.009.

[10] D. Pisinger, "Where are the hard knapsack problems?" *Comput. Oper. Res.*, vol. 32, no. 9, pp. 2271–2284, 2005, doi: 10.1016/j.cor.2004.03.002.

[11] J. C. Ortiz-Bayliss, E. Özcan, A. J. Parkes, and H. Terashima-Marín, "A genetic programming hyper-heuristic: Turning features into heuristics for constraint satisfaction," in *Proc. UKCI*, 2013, pp. 183–190, doi: 10.1109/UKCI.2013.6651304.

[12] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle, and M. Birattari, "The irace package: Iterated racing for automatic algorithm configuration," *Oper. Res. Perspect.*, vol. 3, pp. 43–58, 2016, doi: 10.1016/j.orp.2016.09.002.

[13] M. López-Ibáñez, J. Branke, and L. Paquete, "Reproducibility in evolutionary computation," *ACM Trans. Evol. Learn. Optim.*, vol. 1, no. 4, Art. no. 14, pp. 1–21, 2021, doi: 10.1145/3466624.